

MODULE 5

Durcir le navigateur et le canal

Module 5. Durcir le navigateur et le canal

Ce module durcit la surface HTTP de TaskFlow : on active Helmet, on protège les formulaires contre le CSRF, on verrouille les cookies, et on force HTTPS en local.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application : - ☒ Helmet configuré avec CSP, HSTS, X-Frame-Options - ☒ Token CSRF généré et vérifié sur toutes les routes POST non-API - ☒ Cookies session avec flags `HttpOnly` , `Secure` , `SameSite=Strict` - ☒ Certificat local mkcert + redirection HTTP→HTTPS - ☒ Section M5 rédigée dans le rapport

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
Rappels	Concepts 1-8	Lecture active	15 min
1	Helmet + CSP	<code>npm install helmet</code> + config	10 min
2	CSRF token	<code>npm install csurf</code> + middleware	15 min
3	Durcissement cookies	Flags session	5 min
4	HTTPS local	mkcert + redirect	10 min
Quiz	QCM 6 questions	,	5 min

CONCEPT 1, CSRF (Cross-Site Request Forgery)

Rappel. Un site malveillant fait exécuter une requête HTTP à un utilisateur déjà authentifié sur ton application, en exploitant le fait que le navigateur envoie automatiquement les cookies de session. L'attaquant ne voit pas la réponse, mais l'action est exécutée : transfert d'argent, changement d'email, suppression de données. Le serveur ne peut pas distinguer une requête légitime d'une requête forgée si aucun token secret n'est vérifié.

Risque concret	Un utilisateur connecté sur TaskFlow visite un site piégé. Une balise <code></code> ou un formulaire caché déclenche <code>POST /tasks/delete</code> en son nom. La tâche est supprimée sans que l'utilisateur ait rien fait.
Exemple de faille	Voir bloc ci-dessous
À vérifier dans l'app	Aucun middleware CSRF présent ; toutes les routes POST acceptent n'importe quelle origine
Correction	Voir bloc ci-dessous
Test de validation	<code>curl -X POST</code> <code>http://localhost:3000/tasks/1/delete</code> sans token CSRF doit retourner <code>403 Forbidden</code>
Trace rapport	« Aucun mécanisme CSRF n'était présent sur les routes POST. Le middleware <code>csrf</code> a été ajouté ; chaque formulaire HTML inclut désormais un champ <code>_csrf</code> signé côté serveur. »

Code vulnérable :

```
// vuln-app/routes/tasks.js, AVANT
router.post('/tasks/:id/delete', requireAuth, (req, res) => {
  db.run('DELETE FROM tasks WHERE id = ?', [req.params.id]);
  res.redirect('/tasks');
});
// Aucun contrôle d'origine, aucun token, n'importe quel site peut déclencher cette route.
```

Code corrigé :

```
// app.js, ajouter APRÈS express-session
const csrf = require('csrf');
const csrfProtection = csrf({ cookie: false }); // token lié à la session

app.use(csrfProtection);

// Injecter le token dans toutes les vues
app.use((req, res, next) => {
  res.locals.csrfToken = req.csrfToken();
  next();
});
```

```
<!-- Dans chaque formulaire HTML -->
<form method="POST" action="/tasks/1/delete">
  <input type="hidden" name="_csrf" value="<%= csrfToken %>">
  <button type="submit">Supprimer</button>
</form>
```

TP, Étape 2 : Ajouter CSRF

ÉTAPE 2.1, INSTALLATION

```
cd vuln-app
npm install csrf
```

ÉTAPE 2.2, INTÉGRATION DANS APP.JS

Ouvrir `vuln-app/app.js` . Après la configuration de `express-session` , ajouter le bloc corrigé ci-dessus.

ÉTAPE 2.3, TEST DE VALIDATION

```
# Sans token : doit retourner 403
curl -X POST http://localhost:3000/tasks/1/delete \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -b "connect.sid=<session_valide>"
```

✓ **Point de vérification** : réponse `403 invalid csrf token` .

CONCEPT 2, Token CSRF Synchronizer Pattern

Rappel. Le pattern synchronizer génère un token aléatoire unique par session côté serveur, l'insère dans le HTML rendu, et l'exige dans chaque requête POST/PUT/DELETE. Un site tiers ne peut pas lire le token (Same-Origin Policy) donc ne peut pas forger la requête. Ce pattern est implémenté par `csrf` . Il faut distinguer : les routes API JSON (protégées par vérification du header `Content-Type` et de l'absence de cookies cross-origin) et les routes HTML (protégées par le token dans le formulaire).

Risque concret	Sans token, un formulaire <code><form></code> hébergé sur <code>evil.com</code> peut soumettre à <code>taskflow.local</code> en utilisant les cookies de la victime.
Exemple de faille	Template EJS sans <code>_csrf</code> , formulaire accepté inconditionnellement
À vérifier dans l'app	Chercher tous les <code><form method="POST"></code> dans <code>vuln-app/views/</code> , aucun n'a de champ <code>_csrf</code>
Correction	Ajouter <code><input type="hidden" name="_csrf" value="<%= csrfToken %>"></code> dans chaque formulaire
Test de validation	Soumettre le formulaire sans le champ caché → <code>403</code>
Trace rapport	« Le pattern synchronizer a été implémenté via <code>csrf</code> . Tous les formulaires HTML embarquent un token signé HMAC, vérifié côté serveur avant traitement. »

Audit des templates :

```
grep -rn 'method="POST"' vuln-app/views/
# Vérifier que chaque occurrence a <input type="hidden" name="_csrf"
```

CONCEPT 3, SameSite Cookie

Rappel. L'attribut `SameSite` d'un cookie indique au navigateur dans quels contextes cross-site il doit envoyer ce cookie. `Strict` : le cookie n'est jamais envoyé lors d'une navigation cross-site (protection CSRF maximale). `Lax` : le cookie est envoyé uniquement lors de navigations GET de haut niveau (liens). `None` : toujours envoyé (nécessite `Secure`). Pour une application qui n'a pas besoin d'être embarquée dans des iframes tiers, `Strict` est la valeur recommandée.

Risque concret	Cookie de session envoyé sur toutes les requêtes cross-site → CSRF facilité même avec un token si implémentation incomplète
Exemple de faille	<code>app.use(session({ secret: 'abc', cookie: {} })))</code> , pas de SameSite
À vérifier dans l'app	<code>grep -n "cookie" vuln-app/app.js</code> , vérifier absence de <code>sameSite</code>
Correction	<code>cookie: { sameSite: 'strict', httpOnly: true, secure: true }</code>
Test de validation	Inspecter les headers Set-Cookie dans Firefox DevTools → <code>SameSite=Strict</code> visible
Trace rapport	« L'attribut <code>SameSite=Strict</code> a été ajouté au cookie de session, réduisant le vecteur CSRF au niveau navigateur. »

Code corrigé :

```
// vuln-app/app.js
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,      // Concept 4
    secure: true,        // Concept 4
    sameSite: 'strict',  // ← ajout
    maxAge: 3600000      // 1h
  }
}));
```

CONCEPT 4, HttpOnly / Secure Flags

Rappel. `HttpOnly` interdit à JavaScript (y compris un script XSS injecté) de lire le cookie via `document.cookie`. `Secure` force le navigateur à n'envoyer le cookie que sur une connexion HTTPS. Ces deux flags sont indépendants mais complémentaires : `HttpOnly` protège contre le vol de session via XSS, `Secure` protège contre l'interception réseau (man-in-the-middle). En développement local avec `mkcert`, les deux peuvent être activés simultanément.

Risque concret	Sans <code>HttpOnly</code> : un script XSS peut voler le cookie de session (<code>document.cookie</code>). Sans <code>Secure</code> : le cookie transite en clair sur HTTP.
Exemple de faille	<code>cookie: {}</code> , aucun flag défini, cookie lisible par JS et envoyé en HTTP
À vérifier dans l'app	<code>curl -I http://localhost:3000/login</code> → vérifier <code>Set-Cookie</code> dans la réponse
Correction	<code>cookie: { httpOnly: true, secure: true, sameSite: 'strict' }</code> (voir Concept 3)
Test de validation	Ouvrir la console navigateur : <code>document.cookie</code> ne doit pas afficher le cookie de session
Trace rapport	« Les flags <code>HttpOnly</code> et <code>Secure</code> ont été activés sur le cookie de session, bloquant le vol via XSS et l'interception réseau. »

TP, Étape 4 : Durcir les cookies

```
# Avant correction, vérifier les flags absents
curl -I http://localhost:3000/login
# Chercher "Set-Cookie" dans la réponse
```

Modifier `vuln-app/app.js` avec la configuration de session du Concept 3.

```
# Après correction
curl -I https://localhost:3000/login
# Set-Cookie doit afficher : HttpOnly; Secure; SameSite=Strict
```

✓ **Point de vérification** : la ligne `Set-Cookie` contient les trois attributs.

CONCEPT 5, HSTS (HTTP Strict Transport Security)

Rappel. HSTS est un header HTTP que le serveur envoie pour indiquer au navigateur de n'utiliser que HTTPS pour ce domaine pendant une durée définie. Même si l'utilisateur tape `http://` , le navigateur refuse la connexion non-chiffrée et force `https://` directement. La valeur `max-age=31536000` correspond à un an. L'option `includeSubDomains` étend la protection aux sous-domaines. Ce header n'a d'effet que sur une connexion HTTPS initiale.

Risque concret	Sans HSTS, un attaquant peut intercepter la première requête HTTP et rediriger vers un site frauduleux (SSL stripping).
Exemple de faille	Aucun header <code>Strict-Transport-Security</code> , vérifié avec <code>curl -I</code>
À vérifier dans l'app	<code>curl -I https://localhost:3000</code> → absence de <code>Strict-Transport-Security</code>
Correction	Helmet active HSTS automatiquement avec <code>hsts: { maxAge: 31536000 }</code>
Test de validation	<code>curl -I https://localhost:3000</code> → header <code>Strict-Transport-Security: max-age=31536000</code> présent
Trace rapport	« Le header HSTS avec <code>max-age=31536000</code> a été activé via Helmet, forçant les connexions futures en HTTPS. »

CONCEPT 6, Content Security Policy (CSP)

Rappel. CSP est un header qui liste les sources autorisées pour chaque type de ressource : scripts, styles, images, iframes. Il bloque les scripts injectés par XSS qui ne proviennent pas d'une source approuvée. `default-src 'self'` interdit le chargement de toute ressource externe. On peut affiner : `script-src 'self' cdn.example.com` pour autoriser un CDN spécifique. La directive `report-uri` permet de collecter les violations sans bloquer (mode `Content-Security-Policy-Report-Only`).

Risque concret	Sans CSP, un script XSS peut charger une bibliothèque depuis un serveur attaquant et exfiltrer des données.
Exemple de faille	Aucun header <code>Content-Security-Policy</code> , tout script externe est accepté
À vérifier dans l'app	<code>curl -I https://localhost:3000</code> → absence de <code>content-security-policy</code>
Correction	Voir configuration Helmet ci-dessous
Test de validation	<code>curl -I https://localhost:3000</code> → header CSP présent avec <code>default-src 'self'</code>
Trace rapport	« Une Content Security Policy restrictive (<code>default-src 'self'</code>) a été déployée via Helmet, réduisant la surface d'exploitation XSS. »

Configuration Helmet avec CSP :

```
// vuln-app/app.js
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self'],
      styleSrc: ['self', 'unsafe-inline'], // à durcir si possible
      imgSrc: ['self', 'data:'],
      connectSrc: ['self'],
      fontSrc: ['self'],
      objectSrc: ['none'],
      frameSrc: ['none'],
    },
  },
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true,
  },
}));
```

TP, Étape 1 : Installer et configurer Helmet

```
cd vuln-app  
npm install helmet
```

Ajouter le bloc de configuration ci-dessus en haut de `vuln-app/app.js`, avant toute route.

```
# Vérifier les headers  
curl -I https://localhost:3000
```

✓ **Point de vérification :** `content-security-policy`, `strict-transport-security`, `x-frame-options` présents dans la réponse.

CONCEPT 7, X-Frame-Options / Clickjacking

Rappel. Le clickjacking consiste à superposer un iframe transparent contenant ton application sur un site piège, et à faire cliquer l'utilisateur sur des éléments invisibles (bouton "supprimer", "confirmer le virement"). Le header `X-Frame-Options: DENY` interdit au navigateur d'afficher la page dans n'importe quel iframe. `SAMEORIGIN` l'autorise uniquement pour les iframes de même origine. Helmet ajoute ce header automatiquement avec la valeur `SAMEORIGIN` par défaut ; pour une application sans besoin d'iframe externe, `DENY` est préférable.

Risque concret	TaskFlow embarqué dans un iframe invisible → l'utilisateur clique sur "Supprimer toutes les tâches" en pensant cliquer sur autre chose.
Exemple de faille	Aucun header <code>X-Frame-Options</code>
À vérifier dans l'app	<code>curl -I https://localhost:3000</code> → chercher <code>x-frame-options</code>
Correction	<code>app.use(helmet.frameguard({ action: 'deny' })))</code> ou inclus dans la config Helmet globale
Test de validation	Créer un fichier HTML local avec <code><iframe src="https://localhost:3000"></code> → le navigateur bloque l'affichage
Trace rapport	« Le header <code>X-Frame-Options: DENY</code> a été activé, bloquant toute tentative de clickjacking via iframe. »

CONCEPT 8, HTTPS Local avec mkcert

Rappel. En développement, les certificats auto-signés génèrent des avertissements navigateur et ne permettent pas de tester les comportements réels (HSTS, Secure cookies). `mkcert` crée une autorité de certification locale reconnue par le système et génère des certificats valides pour `localhost`. Cela permet de tester le comportement de production sans déployer. La redirection HTTP→HTTPS s'implémente avec un middleware qui vérifie `req.secure` ou le header `x-forwarded-proto`.

Risque concret	Sans HTTPS en dev, les flags <code>Secure</code> sur les cookies sont ignorés et le comportement HSTS ne peut pas être testé.
Exemple de faille	Serveur HTTP pur : <code>app.listen(3000)</code> , cookies <code>Secure</code> non envoyés
À vérifier dans l'app	<code>vuln-app/app.js</code> , chercher <code>https.createServer</code> absent
Correction	Voir bloc ci-dessous
Test de validation	<code>curl -k https://localhost:3443</code> répond normalement ; <code>curl http://localhost:3000</code> retourne <code>301 Moved Permanently</code>
Trace rapport	« HTTPS local activé avec mkcert. La redirection HTTP→HTTPS (301) est fonctionnelle. Les flags <code>Secure</code> et HSTS sont effectifs. »

Installation mkcert et génération du certificat :

```
# Windows (PowerShell admin) ou macOS/Linux
# Windows : choco install mkcert OU winget install FiloSottile.mkcert
# macOS : brew install mkcert

mkcert -install          # installe l'autorité CA locale
cd vuln-app
mkcert localhost 127.0.0.1 ::1
# Génère : localhost+2.pem  localhost+2-key.pem
```

Serveur HTTPS dans app.js :

```
// vuln-app/app.js
const https = require('https');
const http = require('http');
const fs = require('fs');

const httpsOptions = {
  key: fs.readFileSync('./localhost+2-key.pem'),
  cert: fs.readFileSync('./localhost+2.pem'),
};

// Redirection HTTP → HTTPS
const httpApp = express();
httpApp.use((req, res) => {
  res.redirect(301, `https://${req.hostname}:3443${req.url}`);
});
http.createServer(httpApp).listen(3000, () => {
  console.log('HTTP redirect listening on port 3000');
});

// Serveur HTTPS principal
https.createServer(httpsOptions, app).listen(3443, () => {
  console.log('HTTPS server running on https://localhost:3443');
});
```

TP, Étape 5 : HTTPS local

```
# Installer mkcert selon votre OS
mkcert -install
cd vuln-app && mkcert localhost 127.0.0.1 ::1

# Redémarrer l'application
node app.js

# Tester la redirection
curl -v http://localhost:3000/
# Doit retourner : 301 Moved Permanently → https://localhost:3443

# Tester HTTPS
curl -k -I https://localhost:3443/
# Doit retourner tous les headers Helmet
```

✓ **Point de vérification :** `curl -I https://localhost:3443` affiche :

```
Strict-Transport-Security: max-age=31536000
Content-Security-Policy: default-src 'self' ...
X-Frame-Options: DENY
```

Capture attendue, Récapitulatif headers

Après toutes les étapes :

```
curl -I https://localhost:3443
```

Résultat attendu :

```
HTTP/2 200
content-security-policy: default-src 'self'; script-src 'self'; ...
strict-transport-security: max-age=31536000; includeSubDomains; preload
x-frame-options: DENY
x-content-type-options: nosniff
x-permitted-cross-domain-policies: none
referrer-policy: no-referrer
```

Mini-quiz M5 (6 QCM)

- Q1.** Que fait le flag `HttpOnly` sur un cookie ? - A. Il force le cookie à n'être envoyé qu'en HTTPS - B. Il interdit à JavaScript de lire le cookie - C. Il empêche le cookie d'être envoyé cross-site - D. Il fixe la durée de vie du cookie
- Q2.** Quelle valeur de `SameSite` offre la protection CSRF maximale ? - A. None - B. Lax - C. Strict - D. Default
- Q3.** HSTS protège contre : - A. Les injections SQL - B. Le SSL stripping et les connexions HTTP non chiffrées - C. Le vol de cookies via XSS - D. Les attaques DDoS
- Q4.** Que bloque `X-Frame-Options: DENY` ? - A. Les requêtes cross-origin - B. L'affichage de la page dans un iframe quel que soit le contexte - C. Les scripts non-inline - D. Les connexions HTTP
- Q5.** Dans le pattern synchronizer CSRF, où est stocké le token côté serveur ? - A. Dans la base de données - B. Dans un cookie séparé - C. Dans la session utilisateur - D. Dans l'URL
- Q6.** `default-src 'self'` dans une CSP signifie : - A. Seuls les scripts signés sont acceptés - B. Toutes les ressources doivent provenir de la même origine - C. Les scripts inline sont autorisés - D. Les images externes sont bloquées uniquement

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	<code>HttpOnly</code> bloque l'accès via <code>document.cookie</code>
Q2	C	<code>Strict</code> n'envoie jamais le cookie en contexte cross-site
Q3	B	HSTS force HTTPS dès la prochaine visite, bloquant le downgrade
Q4	B	DENY interdit tous les iframes, même same-origin
Q5	C	Le token est lié à la session serveur, impossible à lire cross-site
Q6	B	<code>'self'</code> = même schéma, même hôte, même port

Livrable du module

- ☐ `npm install helmet csurf` effectué
- ☐ Helmet configuré avec CSP, HSTS, X-Frame-Options dans `app.js`
- ☐ Middleware CSRF ajouté, token présent dans tous les formulaires HTML
- ☐ Cookies session avec `httpOnly: true`, `secure: true`, `sameSite: 'strict'`
- ☐ mkcert installé, certificat local généré, serveur HTTPS sur port 3443
- ☐ `curl -I https://localhost:3443` montre tous les headers de sécurité
- ☐ Captures dans `proofs/M5/`
- ☐ Tag Git : `git tag M5-headers-csrf && git push --tags`
- ☐ Section M5 rédigée dans le rapport

Erreurs fréquentes à éviter

- Activer `secure: true` sur les cookies sans activer HTTPS → les cookies ne sont jamais envoyés.
- Oublier d'injecter `res.locals.csrfToken` dans un middleware global → certaines vues n'ont pas accès au token.

- Utiliser `SameSite: None` sans `Secure` → rejeté par les navigateurs modernes.
- Configurer CSP trop stricte qui casse les styles → tester d'abord en mode `Content-Security-Policy-Report-Only`.
- Committer les fichiers `.pem` de mkcert dans le dépôt → les ajouter à `.gitignore`.

Ressources

- OWASP Top 10, A05:2021 Security Misconfiguration : https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- OWASP CSRF Prevention Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- Helmet.js documentation : <https://helmetjs.github.io/>
- mkcert : <https://github.com/FiloSottile/mkcert>
- MDN, Content Security Policy : <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>